



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

título del TFG



Presentado por Álvaro Pérez Mella
en Universidad de Burgos — 15 de abril
de 2026

Tutores: Jose Luis Garrido Labrador y Jose
Miguel Ramirez Sanz



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. nombre tutor, profesor del departamento de nombre departamento, área de nombre área.

Expone:

Que el alumno D. Álvaro Pérez Mella, con DNI dni, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado título de TFG.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 15 de abril de 2026

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

D. nombre tutor

D. nombre co-tutor

Resumen

En este primer apartado se hace una **breve** presentación del tema que se aborda en el proyecto.

Descriptores

Palabras separadas por comas que identifiquen el contenido del proyecto Ej: servidor web, buscador de vuelos, android ...

Abstract

A **brief** presentation of the topic addressed in the project.

Keywords

keywords separated by commas.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
1. Introducción	1
2. Objetivos del proyecto	3
2.1. Objetivo general	3
2.2. Objetivos del software	3
2.3. Objetivos técnicos	4
3. Conceptos teóricos	5
3.1. Visión por computador y procesamiento de imágenes	5
3.2. Detección de objetos y análisis mediante puntos clave	6
3.3. YOLO como modelo de detección de objetos	7
3.4. Conceptos asociados a la salida de YOLO	7
3.5. MediaPipe como framework de percepción	8
3.6. Funcionamiento interno de MediaPipe en manos y pose	9
3.7. Diferencia entre detección por cajas y detección por landmarks	9
3.8. Procesamiento de imágenes en una aplicación web	10
3.9. Persistencia y modelo de datos	10
3.10. Autenticación y protección de recursos	11
3.11. Síntesis	11
4. Técnicas y herramientas	13

4.1. Visión por Computador e Inteligencia Artificial	13
4.2. Desarrollo Backend y Lógica de Negocio	14
4.3. Gestión de Datos	15
4.4. Diseño y Diagramación	15
4.5. Documentación y redacción técnica	15
4.6. Entorno de Desarrollo e Infraestructura	15
4.7. Gestión y Comunicación del Proyecto	16
4.8. Asistencia mediante IA Generativa	16
5. Aspectos relevantes del desarrollo del proyecto	19
5.1. Enfoque general y ciclo de vida	19
5.2. Planteamiento inicial del problema	20
5.3. Elección tecnológica: Flask y ecosistema Python	20
5.4. Selección de modelos para la demo	21
5.5. Construcción de la demo	21
5.6. Decisiones de diseño relevantes en la demo	22
5.7. Aspectos relevantes de implementación	23
5.8. Limitaciones de la demo y necesidad de evolución	26
5.9. De la demo a la aplicación real	26
5.10. Diseño de datos y persistencia	26
5.11. Migración del entorno de desarrollo a Debian	27
5.12. Implementación de la base de datos en MariaDB	27
5.13. Síntesis	27
6. Trabajos relacionados	29
7. Conclusiones y Líneas de trabajo futuras	31
Bibliografía	33

Índice de figuras

5.1. Resultado de la ejecución de MediaPipe en modo hands , con representación de landmarks y conexiones sobre ambas manos detectadas.	23
5.2. Capturas asociadas a MediaPipe en modo pose . En (a) se muestra la interfaz de la demo con la imagen de entrada y los landmarks corporales detectados. En (b) se presenta la salida JSON generada, con información de landmarks, visibilidad y ruta de la imagen procesada.	24
5.3. Capturas asociadas al modelo YOLO. En (a) se observa la interfaz de la demo mostrando la imagen procesada con cajas delimitadoras sobre los objetos detectados. En (b) se presenta la salida JSON correspondiente, incluyendo detecciones, confianza, coordenadas y ruta de la imagen generada.	25

Índice de tablas

1. Introducción

Descripción del contenido del trabajo y del estructura de la memoria y del resto de materiales entregados.

2. Objetivos del proyecto

El presente Trabajo de Fin de Grado tiene como finalidad el desarrollo de una aplicación web orientada al procesamiento de imágenes mediante modelos de inteligencia artificial. La aplicación deberá permitir enviar una imagen al servidor, ejecutar sobre ella un modelo de análisis y devolver un resultado útil tanto desde el punto de vista funcional como visual.

A partir de esta finalidad general, pueden distinguirse dos tipos de objetivos: por un lado, los objetivos relacionados con el software que se desea construir y, por otro, los objetivos técnicos necesarios para llevar a cabo su diseño e implementación.

2.1. Objetivo general

El objetivo general del proyecto es desarrollar una aplicación web capaz de ejecutar distintos modelos de inteligencia artificial sobre imágenes, devolviendo como resultado información estructurada y una imagen procesada.

2.2. Objetivos del software

Los principales objetivos del software a desarrollar son los siguientes:

- Permitir la carga de imágenes por parte del usuario para su procesamiento.
- Integrar distintos modelos de inteligencia artificial dentro de una misma aplicación.

- Ofrecer resultados comprensibles, tanto en formato estructurado como visual.
- Diferenciar la interfaz web de uso interactivo de la API destinada a pruebas o integración.
- Diseñar una base funcional que pueda ampliarse en el futuro con nuevos modelos, modos de ejecución y otras funcionalidades.

2.3. Objetivos técnicos

Junto a los objetivos funcionales, el proyecto plantea varios objetivos técnicos:

- Diseñar una arquitectura modular que facilite la organización y evolución del sistema.
- Emplear tecnologías adecuadas para el desarrollo de aplicaciones web con procesamiento de imágenes e inteligencia artificial.
- Definir un modelo de datos que permita incorporar persistencia y soporte para futuras ampliaciones.
- Implementar una base técnica mantenible y coherente con un posible despliegue real.

En conjunto, el proyecto busca no sólo construir una aplicación funcional, sino también establecer una base técnica sólida sobre la que pueda apoyarse su evolución posterior.

3. Conceptos teóricos

En aquellos proyectos que necesiten para su comprensión y desarrollo de unos conceptos teóricos de una determinada materia o de un determinado dominio de conocimiento, debe existir un apartado que sintetice dichos conceptos.

Para comprender adecuadamente el desarrollo del presente TFG es necesario introducir una serie de conceptos teóricos relacionados con la visión por computador, el procesamiento de imágenes y la construcción de aplicaciones web capaces de integrar modelos de inteligencia artificial. Este capítulo no pretende realizar una exposición exhaustiva de cada tecnología empleada, sino sintetizar aquellos fundamentos que permiten entender el problema abordado y las decisiones tomadas durante el desarrollo.

3.1. Visión por computador y procesamiento de imágenes

La visión por computador es una rama de la inteligencia artificial orientada a extraer información útil a partir de imágenes o secuencias de vídeo. Su objetivo no se limita a almacenar o mostrar imágenes, sino a interpretarlas computacionalmente para detectar objetos, localizar puntos clave, reconocer patrones o describir el contenido visual de una escena.

Desde el punto de vista del tratamiento interno, una imagen digital puede entenderse como una matriz de píxeles. Cada píxel almacena información de intensidad o color y, en imágenes en color, suele representarse mediante varios canales. Bibliotecas como OpenCV permiten cargar, transformar, anotar y

guardar imágenes de forma eficiente, facilitando además su integración con estructuras matriciales manejadas en Python [14].

En aplicaciones de visión por computador, esta representación matricial resulta esencial, ya que los modelos de inteligencia artificial operan sobre la información numérica contenida en la imagen. Por ello, antes de aplicar un modelo, la imagen debe ser decodificada, adaptada al formato esperado y, tras el procesamiento, convertida nuevamente en una salida interpretable por el usuario.

3.2. Detección de objetos y análisis mediante puntos clave

Dentro de la visión por computador existen distintos tipos de tareas. Dos de las más relevantes para este proyecto son la **detección de objetos** y la **detección de puntos clave o landmarks**.

La detección de objetos consiste en localizar en una imagen uno o varios elementos de interés y asignarles una clase. El resultado suele expresarse mediante cajas delimitadoras (*bounding boxes*), una etiqueta identificativa y una puntuación de confianza. Este tipo de salida resulta especialmente útil cuando se desea identificar la presencia y posición aproximada de objetos dentro de una escena.

Por su parte, la detección de landmarks no se centra en delimitar un objeto completo mediante una caja, sino en localizar puntos anatómicos o estructurales relevantes. En modelos aplicados al cuerpo humano o a las manos, estos puntos pueden corresponder a articulaciones, extremos o regiones significativas. El valor de este enfoque es que ofrece una descripción más rica de la geometría del objeto analizado, permitiendo representar posturas, relaciones espaciales y conexiones entre puntos.

Ambos enfoques son complementarios. Mientras la detección de objetos proporciona una descripción general basada en localización y clasificación, los landmarks permiten un análisis más detallado de la estructura interna de ciertos elementos de la imagen.

3.3. YOLO como modelo de detección de objetos

YOLO (*You Only Look Once*) es una familia de modelos de detección de objetos caracterizada por su enfoque de una sola etapa (*one-stage detector*). Frente a otros métodos más complejos que separan la propuesta de regiones y la clasificación en varias fases, YOLO realiza la detección en una única pasada sobre la imagen, lo que lo convierte en una solución especialmente adecuada para escenarios donde se busca rapidez de inferencia y resultados visuales inmediatos [16].

De forma general, el funcionamiento interno de un detector YOLO puede resumirse en varias etapas. En primer lugar, la imagen se introduce en una red neuronal convolucional capaz de extraer características visuales relevantes. A partir de esas características, el modelo predice directamente posibles cajas delimitadoras, probabilidades de clase y valores de confianza. Posteriormente, se aplica un proceso de filtrado para eliminar detecciones redundantes o poco fiables, de manera que el resultado final conserve únicamente las detecciones más representativas.

Una de las ventajas principales de YOLO es que integra localización y clasificación en un mismo flujo de cálculo. Esto reduce la complejidad del procesamiento y favorece el rendimiento en tiempo real o casi real. Además, su salida es fácilmente interpretable por el usuario, ya que muestra de forma visual los objetos detectados y su posición dentro de la imagen.

En el contexto de este TFG, YOLO resulta especialmente útil porque permite validar de manera clara el procesamiento de imágenes mediante inteligencia artificial. La aparición de cajas, etiquetas y niveles de confianza sobre la imagen constituye una evidencia visual directa del funcionamiento del sistema.

3.4. Conceptos asociados a la salida de YOLO

Para interpretar correctamente la salida de un detector de objetos conviene tener en cuenta algunos conceptos básicos.

En primer lugar, cada detección suele ir acompañada de una **clase**, que indica el tipo de objeto reconocido, y de una **confianza**, que expresa el grado de seguridad del modelo respecto a esa predicción. Además, la

localización del objeto se representa mediante una caja delimitadora definida por coordenadas dentro de la imagen.

Cuando el modelo produce varias detecciones solapadas sobre una misma región, es habitual aplicar mecanismos de supresión de redundancia, de forma que sólo se conserve la detección más representativa. Este proceso mejora la claridad del resultado final y evita que un mismo objeto aparezca marcado varias veces.

Por tanto, la salida de YOLO no debe entenderse únicamente como una imagen con rectángulos superpuestos, sino como una representación estructurada del contenido visual en la que cada detección combina localización, clasificación y confianza [16].

3.5. MediaPipe como framework de percepción

MediaPipe es un framework orientado al desarrollo de soluciones de percepción multimodal en tiempo real, especialmente útil en tareas de visión por computador relacionadas con rostro, manos, pose corporal y otros elementos anatómicos [8, 3].

A diferencia de un modelo aislado concebido únicamente para producir una predicción, MediaPipe se plantea como una infraestructura capaz de organizar flujos completos de procesamiento. Internamente, su funcionamiento se apoya en grafos de cálculo formados por nodos o componentes que reciben datos, ejecutan una operación y entregan su salida al siguiente nodo. Esta organización facilita combinar distintas etapas dentro de un mismo pipeline, como preprocesado, detección, seguimiento, estimación de landmarks y postprocesado [8].

Esta estructura por grafos representa una de las características más importantes de MediaPipe, ya que permite diseñar soluciones modulares y eficientes. En lugar de concebir el procesamiento como un bloque monolítico, el framework lo divide en operaciones conectadas entre sí, favoreciendo la reutilización y la adaptación a distintos casos de uso.

3.6. Funcionamiento interno de MediaPipe en manos y pose

En soluciones como *Hands* o *Pose*, MediaPipe combina normalmente varias fases. De forma simplificada, primero localiza la región de interés donde previsiblemente se encuentra la mano o el cuerpo, y posteriormente aplica un modelo más específico para estimar los landmarks dentro de esa región [3].

En el caso de las manos, el sistema no se limita a indicar que existe una mano en la imagen, sino que estima puntos clave correspondientes a distintas articulaciones y extremos. A partir de esos puntos, es posible dibujar conexiones anatómicas y obtener una representación estructurada de la mano.

De forma análoga, en la estimación de pose corporal, MediaPipe detecta puntos clave distribuidos por distintas regiones del cuerpo. La salida no consiste en una simple caja envolvente, sino en un conjunto de coordenadas asociadas a landmarks corporales. En muchos casos, cada landmark puede incluir además atributos adicionales, como visibilidad o presencia, que ayudan a interpretar la fiabilidad del resultado [3].

Este tipo de análisis aporta una información mucho más rica que la detección general de objetos. Mientras un detector clásico puede indicar dónde está una persona, un modelo basado en landmarks permite describir su postura o la disposición relativa de sus extremidades.

3.7. Diferencia entre detección por cajas y detección por landmarks

Desde el punto de vista teórico, la diferencia entre YOLO y MediaPipe en este proyecto resulta especialmente ilustrativa.

YOLO trabaja principalmente con el paradigma de **detección de objetos**: identifica entidades presentes en la imagen y las delimita mediante cajas. Su salida es adecuada cuando interesa conocer qué objetos aparecen y dónde se localizan de manera aproximada.

MediaPipe, en cambio, se orienta a la **estimación estructural mediante landmarks**. En lugar de ofrecer una delimitación global del objeto, describe puntos internos significativos y sus relaciones espaciales. Esta

aproximación resulta especialmente adecuada cuando interesa analizar configuraciones anatómicas, posturas o movimientos.

Por tanto, ambos enfoques responden a necesidades distintas dentro de la visión por computador. La combinación de ambos en un mismo proyecto permite cubrir tipos de análisis complementarios y demuestra la flexibilidad de la aplicación desarrollada.

3.8. Procesamiento de imágenes en una aplicación web

Cuando un modelo de visión por computador se integra dentro de una aplicación web, el flujo de trabajo implica varias etapas. En primer lugar, el cliente envía una imagen al servidor. Después, el servidor la recibe, la valida, la transforma a una representación interna adecuada y ejecuta sobre ella el modelo seleccionado. Finalmente, el sistema devuelve una respuesta que puede incluir tanto información estructurada como una imagen procesada.

Este planteamiento requiere separar adecuadamente la lógica de comunicación web de la lógica específica de procesamiento. En este contexto, Flask resulta una opción apropiada por su ligereza y por su capacidad para construir servicios web de forma flexible [13]. A través de rutas HTTP, una aplicación basada en Flask puede recibir peticiones, procesar datos y devolver respuestas en distintos formatos, como HTML o JSON.

La separación entre una API orientada a integración y una interfaz web orientada a interacción humana constituye además una decisión coherente desde el punto de vista arquitectónico, ya que evita mezclar responsabilidades y facilita la evolución del sistema.

3.9. Persistencia y modelo de datos

Aunque el procesamiento puntual de imágenes puede realizarse sin almacenamiento persistente, una aplicación real requiere un modelo de datos que permita representar usuarios, permisos, modelos disponibles, configuraciones y ejecuciones realizadas.

En este contexto, los sistemas gestores de bases de datos relacionales permiten estructurar la información mediante tablas, claves primarias, claves foráneas y restricciones de integridad. MariaDB constituye una solución

adecuada para este tipo de aplicaciones por su madurez, soporte transaccional e integración habitual en entornos Linux [4].

Además, el uso de una capa de abstracción como SQLAlchemy facilita la interacción entre la aplicación Python y la base de datos, permitiendo modelar entidades y relaciones de forma más mantenible [2]. De esta manera, la persistencia deja de ser un mero almacenamiento de datos aislados y pasa a reflejar parte de la lógica estructural del sistema.

3.10. Autenticación y protección de recursos

En aplicaciones web que diferencian usuarios, permisos o acceso a determinadas funcionalidades, resulta necesario incorporar mecanismos de autenticación. Uno de los enfoques habituales en APIs consiste en utilizar tokens, de forma que el servidor pueda verificar la identidad del cliente y decidir si una determinada petición está autorizada.

En el ecosistema de Flask, bibliotecas como Flask-JWT-Extended permiten implementar este tipo de mecanismos basados en JWT (*JSON Web Tokens*), facilitando la protección de rutas y la gestión de sesiones de acceso en servicios web [15]. Este concepto resulta relevante en el proyecto porque la evolución prevista de la aplicación exige controlar qué usuarios pueden acceder a qué modelos o modos de ejecución.

3.11. Síntesis

Los conceptos teóricos expuestos en este capítulo permiten situar el proyecto dentro de un marco técnico claro. Por un lado, la visión por computador proporciona la base para comprender el procesamiento de imágenes y la diferencia entre detección de objetos y análisis mediante landmarks. Por otro, modelos y frameworks como YOLO y MediaPipe representan dos enfoques complementarios para extraer información visual de una imagen.

A su vez, la integración de estos modelos dentro de una aplicación web exige comprender nociones relacionadas con el intercambio cliente-servidor, la persistencia de datos y la protección de recursos. En conjunto, estos fundamentos teóricos permiten entender mejor tanto el alcance del proyecto como las decisiones adoptadas en su desarrollo.

4. Técnicas y herramientas

En este capítulo se describen las tecnologías y herramientas seleccionadas para el desarrollo del proyecto. La elección de este *stack* tecnológico responde a un análisis de eficiencia, seguridad y compatibilidad técnica, priorizando soluciones de código abierto y ampliamente respaldadas por la comunidad profesional.

4.1. Visión por Computador e Inteligencia Artificial

La capacidad de procesamiento visual es el núcleo de este proyecto. Se han seleccionado herramientas que permiten un equilibrio entre precisión y rendimiento en tiempo real:

- **MediaPipe:** Framework de Google especializado en la detección de puntos clave corporales. *Justificación:* A diferencia de modelos basados puramente en aprendizaje profundo que requieren GPUs de alto rendimiento, MediaPipe está optimizado para ejecutarse de forma eficiente en CPU mediante el uso de grafos de procesamiento. Esto garantiza que la aplicación sea accesible en una mayor variedad de dispositivos sin sacrificar la latencia [8, 3].
- **YOLO (You Only Look Once) v8:** Modelo de detección de objetos por regresión. *Justificación:* Se ha optado por la versión v8 de Ultralytics por ser el estándar actual en velocidad. Mientras que otros modelos como Faster R-CNN requieren dos etapas de procesamiento, YOLO realiza la detección en una sola pasada, lo que resulta crítico

para sistemas que requieren una respuesta inmediata ante la cámara [16].

- **OpenCV (cv2):** Librería de visión artificial por excelencia. *Justificación:* Se utiliza como "pegamento" técnico para la manipulación de imágenes. Su ventaja competitiva reside en su madurez y en la capacidad de manejar flujos de vídeo de forma nativa, permitiendo tareas de preprocesamiento (como el redimensionado o filtrado de ruido) que son costosas de implementar desde cero en Python [14].

4.2. Desarrollo Backend y Lógica de Negocio

Para la construcción de la API y la orquestación de datos, se ha utilizado el ecosistema de Python, seleccionado por su sintaxis clara y su liderazgo en el sector de la IA:

- **Flask:** Microframework web. *Justificación:* Frente a frameworks "todo incluido" como Django, Flask ofrece una arquitectura minimalista. Esto evita la sobrecarga de componentes innecesarios y permite un control total sobre las rutas y la integración de los modelos de IA, facilitando un despliegue más ágil y ligero [13].
- **Flask-SQLAlchemy:** Capa de abstracción de base de datos (ORM). *Justificación:* Su uso es fundamental para la seguridad y el mantenimiento. Al utilizar un ORM, se evita la escritura de consultas SQL manuales, lo que mitiga el riesgo de ataques de inyección SQL y permite interactuar con los datos mediante objetos de Python, facilitando la legibilidad del código [2].
- **Flask-JWT-Extended:** Gestión de autenticación mediante JSON Web Tokens. *Justificación:* Se ha elegido para implementar una arquitectura *stateless* (sin estado). Esto permite que el servidor no tenga que almacenar sesiones en memoria, mejorando la escalabilidad y proporcionando un mecanismo robusto para proteger los *endpoints* privados del sistema [15].
- **Werkzeug y Python-dotenv:** Herramientas de utilidad y configuración. *Justificación:* Werkzeug garantiza la seguridad de las credenciales mediante el *hashing* de contraseñas, asegurando que no se almacenen en texto plano. Por otro lado, `python-dotenv` permite gestionar variables de entorno, manteniendo las claves privadas y credenciales de acceso fuera del código fuente [7].

4.3. Gestión de Datos

- **MariaDB:** Sistema de gestión de bases de datos relacionales. *Justificación:* Se ha seleccionado MariaDB sobre MySQL por su excelente rendimiento en la gestión de hilos y su naturaleza totalmente *open-source*. Es la solución ideal para garantizar la integridad referencial de los datos de usuarios y registros históricos con una alta disponibilidad [4].

4.4. Diseño y Diagramación

Para la fase de planificación visual y modelado arquitectónico, se han empleado las siguientes herramientas:

- **dbdiagram.io:** Herramienta de diseño de bases de datos basada en código. *Justificación:* Permite definir el esquema Entidad-Relación de forma rápida mediante lenguaje descriptivo (DSL). Su uso ha sido clave para iterar el modelo de datos de MariaDB antes de su implementación física, asegurando que todas las relaciones fuesen correctas.
- **Draw.io:** Aplicación de diagramación profesional. *Justificación:* Se ha utilizado para crear los diagramas de arquitectura del sistema, flujos de datos y lógica de los algoritmos de visión. Su versatilidad permite documentar visualmente el proyecto de forma estandarizada.

4.5. Documentación y redacción técnica

- **LaTeX:** Sistema de preparación de documentos de alta calidad. *Justificación:* Se ha utilizado para la redacción de la memoria del TFG por su capacidad para estructurar documentos técnicos de forma rigurosa y profesional. Su uso facilita la gestión automática de capítulos, referencias cruzadas, bibliografía, figuras, tablas y anexos, además de ofrecer un acabado tipográfico especialmente adecuado para trabajos académicos y documentación técnica extensa [17].

4.6. Entorno de Desarrollo e Infraestructura

Esta sección comprende el sistema operativo y las aplicaciones que han permitido la escritura, depuración y control del software:

- **Linux (Debian):** Sistema operativo base. *Justificación:* Elegido por su estabilidad crítica y su gestión avanzada de permisos. Debian proporciona un entorno de servidor estandarizado donde las dependencias de Python y las librerías de IA se ejecutan de forma previsible y segura [12].
- **Visual Studio Code:** Entorno de desarrollo integrado (IDE). *Justificación:* Su potencia reside en su ecosistema de extensiones y su terminal integrada. Facilita la depuración de código en tiempo real y la integración directa con el control de versiones, optimizando el tiempo de desarrollo [10].
- **DBeaver:** Cliente de base de datos universal. *Justificación:* A diferencia de herramientas básicas, DBeaver permite administrar MariaDB de forma visual, ejecutar consultas de prueba y monitorizar el estado de las tablas sin necesidad de usar la línea de comandos, lo que reduce errores humanos en la manipulación de datos.
- **GitHub:** Plataforma de control de versiones distribuido. *Justificación:* Es fundamental para garantizar la trazabilidad del código. Permite mantener un histórico de cambios, trabajar en diferentes ramas de desarrollo y asegurar que el trabajo esté respaldado en la nube [5].

4.7. Gestión y Comunicación del Proyecto

- **Jira:** Gestión del ciclo de vida del software. *Justificación:* Se ha empleado para implementar una metodología ágil. Permite la planificación de *Sprints*, la priorización de tareas en el *Backlog* y el seguimiento detallado del progreso, asegurando que se cumplen los hitos del TFG en el tiempo previsto [1].
- **Microsoft Teams:** Centro de colaboración. *Justificación:* Ha servido como canal de comunicación persistente para reuniones de seguimiento y compartición de documentos técnicos, manteniendo un histórico de las decisiones tomadas durante el desarrollo [9].

4.8. Asistencia mediante IA Generativa

- **ChatGPT y Gemini:** Modelos de lenguaje de gran tamaño (LLM). *Justificación:* Se han utilizado como asistentes técnicos para la optimización de funciones complejas en Python, la depuración de errores de

configuración en Debian y la generación de casos de prueba. Su uso ha permitido acelerar la resolución de problemas técnicos específicos [11, 6].

5. Aspectos relevantes del desarrollo del proyecto

Este capítulo recoge las decisiones más relevantes adoptadas durante el desarrollo del TFG, tanto desde el punto de vista técnico como desde la lógica que ha guiado la evolución del proyecto. No se pretende describir únicamente las tecnologías empleadas, sino justificar las principales elecciones realizadas en las fases de análisis, diseño e implementación.

El desarrollo ha pasado por dos etapas principales. En primer lugar, se construyó una demo funcional para validar la viabilidad técnica del procesamiento de imágenes mediante modelos de inteligencia artificial. Posteriormente, el trabajo evolucionó hacia la aplicación real, abordando la definición de requisitos, la delimitación del producto mínimo viable, el diseño del modelo de datos, la migración del entorno de desarrollo a Debian y la implementación inicial de la base de datos en MariaDB.

5.1. Enfoque general y ciclo de vida

El proyecto se ha desarrollado siguiendo un enfoque **iterativo e incremental**. En lugar de comenzar directamente con una aplicación completa, se optó por construir primero una demo que permitiera comprobar la viabilidad del procesamiento de imágenes, validar el flujo básico de trabajo y detectar problemas tempranos relacionados con dependencias, rutas, formatos y gestión de ficheros.

Esta estrategia permitió reducir incertidumbre en las primeras fases y tomar decisiones posteriores con una base práctica. Una vez validado el

flujo principal, el proyecto pasó a una etapa más formal de análisis y diseño orientada a la construcción de una aplicación real.

5.2. Planteamiento inicial del problema

El objetivo del TFG es desarrollar una aplicación web capaz de actuar como servidor para ejecutar modelos de inteligencia artificial sobre imágenes, devolviendo tanto resultados estructurados como una imagen procesada.

Desde el inicio se definieron varios principios que han guiado el desarrollo:

- **Resultado visual como salida principal**, complementado con información estructurada en JSON.
- **Extensibilidad**, de forma que puedan añadirse nuevos modelos y modos sin rediseñar el núcleo del sistema.
- **Separación de responsabilidades**, diferenciando la API de la interfaz web.
- **Evolución hacia pipelines**, permitiendo en el futuro encadenar varias etapas de procesamiento.

Por tanto, el problema no consistía sólo en ejecutar un modelo sobre una imagen, sino en sentar las bases de una aplicación ampliable, mantenible y preparada para incorporar control de acceso, catálogo de IAs, persistencia y trazabilidad de ejecuciones.

5.3. Elección tecnológica: Flask y ecosistema Python

Para el backend se eligió **Python** junto con **Flask**. Python resultaba una elección natural por su amplio uso en inteligencia artificial y visión por computador, así como por la disponibilidad de bibliotecas maduras para tratamiento de imágenes y ejecución de modelos.

Flask se seleccionó por su ligereza y flexibilidad. En una fase inicial de experimentación resultaba importante trabajar con una herramienta que permitiera construir una API funcional con rapidez, sin imponer una

estructura excesivamente rígida. Al mismo tiempo, Flask permite evolucionar posteriormente hacia una organización más modular basada en rutas, servicios y separación de capas.

Además, el ecosistema de Python facilitó integrar bibliotecas clave como **OpenCV** y **NumPy**, utilizadas para la carga, representación, manipulación y almacenamiento de imágenes.

5.4. Selección de modelos para la demo

Para la demo inicial se eligieron dos familias de modelos complementarias: **YOLO** y **MediaPipe**. La finalidad de esta elección fue validar distintos tipos de procesamiento y obtener salidas visuales claras que permitiesen comprobar de forma inmediata el funcionamiento del sistema.

YOLO se utilizó como modelo de detección general de objetos. Su salida, basada en cajas delimitadoras, etiquetas y niveles de confianza, resulta muy adecuada para una validación rápida del procesamiento.

MediaPipe se incorporó para trabajar con landmarks, concretamente en los modos **hands** y **pose**. Esto permitió demostrar que el sistema no sólo podía manejar detecciones por caja, sino también estructuras más ricas basadas en puntos clave y conexiones anatómicas.

La combinación de ambos modelos permitió validar dos tipos de análisis frecuentes en visión por computador: detección de objetos y detección de landmarks.

5.5. Construcción de la demo

La demo se desarrolló con dos objetivos principales: comprender el flujo técnico completo del procesamiento de imágenes y validar una primera organización interna del sistema.

En esta fase se consiguió implementar un flujo completo de extremo a extremo:

- Subida de una imagen desde el cliente;
- Ejecución del modelo y modo seleccionados;
- Devolución de una salida estructurada;

- Generación de una imagen anotada.

Este resultado permitió comprobar la viabilidad del procesamiento, la correcta comunicación cliente-servidor y la integración de distintos modelos bajo una misma lógica de uso.

A continuación se incluyen algunas capturas representativas del funcionamiento de la demo.

En primer lugar, la Figura 5.1 muestra el resultado de una ejecución de MediaPipe en modo **hands**. Esta captura permite observar la representación de landmarks y conexiones anatómicas sobre las manos detectadas.

Por su parte, la Figura 5.2 recoge dos capturas asociadas a MediaPipe en modo **pose**: la interfaz de la demo durante la ejecución y la salida JSON generada por el sistema.

Finalmente, la Figura 5.3 presenta dos capturas correspondientes al modelo YOLO: la interfaz con la imagen procesada y la salida JSON asociada a la detección.

5.6. Decisiones de diseño relevantes en la demo

Una de las decisiones más importantes fue organizar la lógica de IA mediante tareas especializadas y una factoría central. Esta estructura permitió separar la selección del modelo y del modo de la capa HTTP, evitando condicionales dispersos y facilitando la extensibilidad del sistema.

También se consideró fundamental separar la **API REST** de la **interfaz web**. La API quedó orientada a integración y pruebas, mientras que la interfaz web se mantuvo como punto de acceso visual para demostración y uso interactivo. Esta separación simplificó la organización del proyecto y resulta coherente con la evolución posterior de la aplicación.

Otro aprendizaje importante estuvo relacionado con la gestión de rutas y la entrega de ficheros generados. Fue necesario resolver correctamente la ubicación de los resultados, su almacenamiento y su acceso posterior desde el navegador. Aunque se trata de un aspecto técnico concreto, resultó decisivo para convertir la prueba inicial en una demo realmente utilizable.

Por último, la interfaz se adaptó para reflejar dinámicamente los modelos y modos disponibles. Esta decisión mejoró la experiencia de uso y redujo el acoplamiento entre frontend y backend.

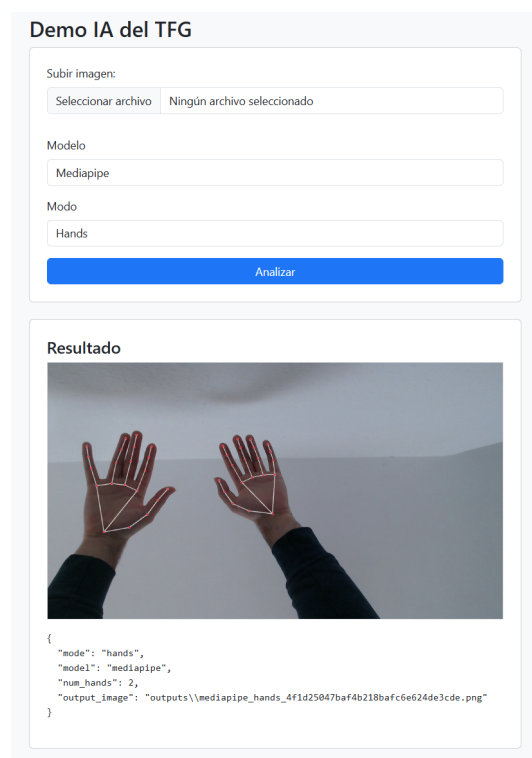
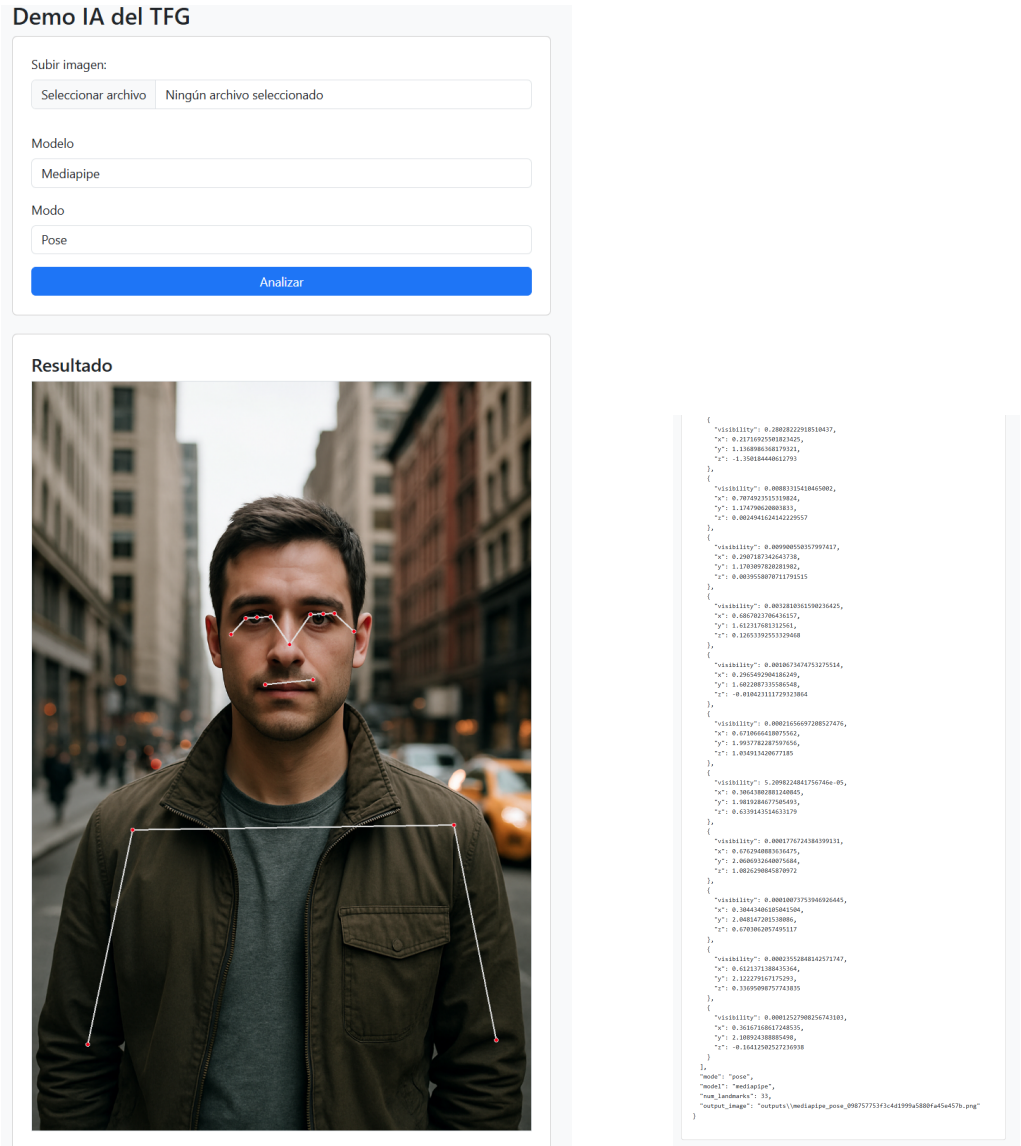


Figura 5.1: Resultado de la ejecución de MediaPipe en modo **hands**, con representación de landmarks y conexiones sobre ambas manos detectadas.

5.7. Aspectos relevantes de implementación

Durante la demo se consolidó un flujo interno común para el tratamiento de imágenes: recepción de la imagen, decodificación a una matriz de **NumPy**, procesamiento con el modelo correspondiente, anotación sobre la imagen y almacenamiento del resultado.

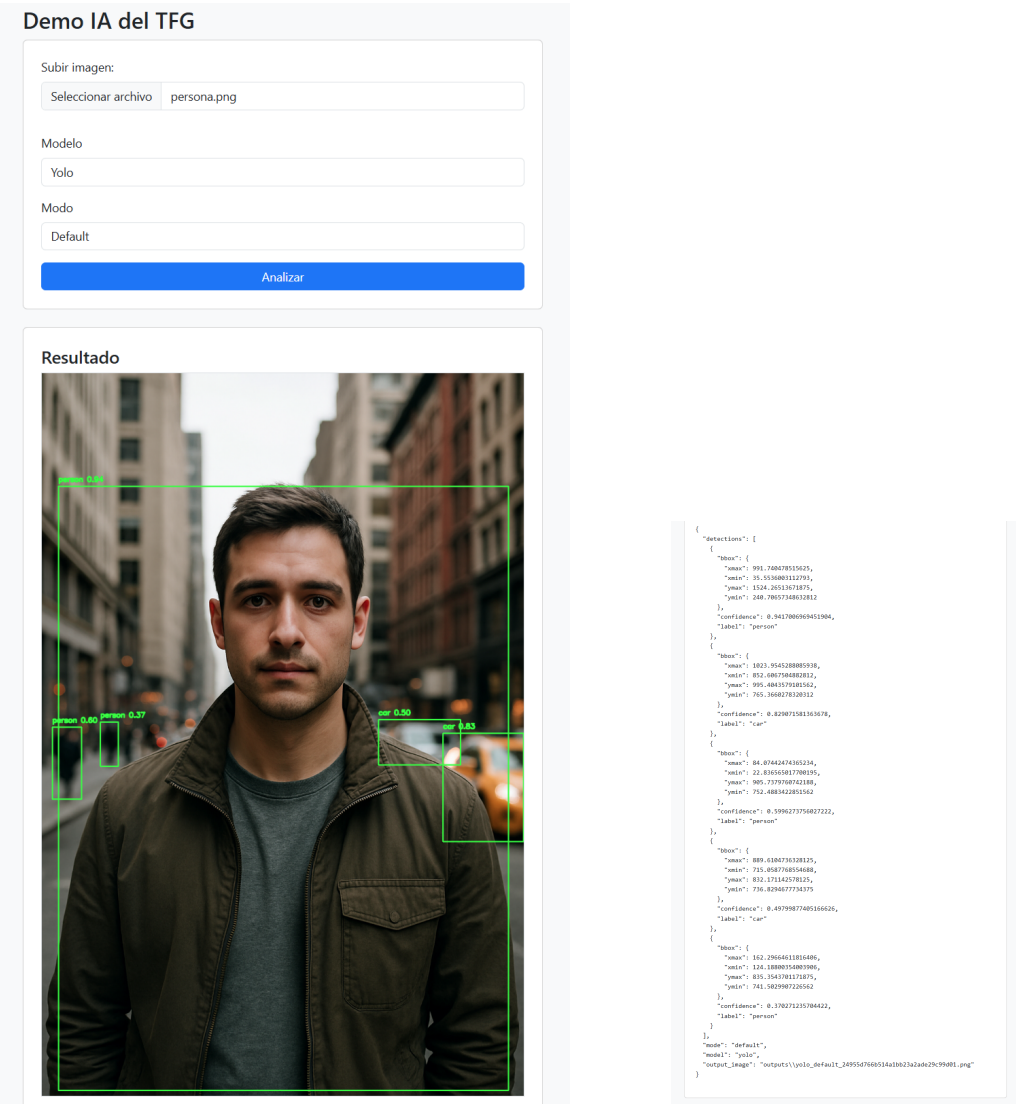
También se estableció una salida dual compuesta por un JSON estructurado y una imagen procesada. Esta combinación aporta, por un lado, trazabilidad e integración y, por otro, una evidencia visual inmediata del funcionamiento del sistema.



(a) Interfaz de la demo durante una ejecución de MediaPipe en modo pose.

(b) Salida JSON generada por MediaPipe en modo pose.

Figura 5.2: Capturas asociadas a MediaPipe en modo pose. En (a) se muestra la interfaz de la demo con la imagen de entrada y los landmarks corporales detectados. En (b) se presenta la salida JSON generada, con información de landmarks, visibilidad y ruta de la imagen procesada.



5.8. Limitaciones de la demo y necesidad de evolución

La demo cumplió su función principal: validar la viabilidad técnica del procesamiento y establecer una primera arquitectura extensible. Sin embargo, seguía siendo un prototipo con limitaciones claras: ausencia de persistencia completa, falta de gestión de usuarios y permisos, trazabilidad reducida, ejecución aislada de tareas y una robustez todavía insuficiente para una aplicación real.

Por ello, la demo no debía entenderse como una versión final reducida, sino como una fase de validación y aprendizaje previa a la construcción formal del sistema.

5.9. De la demo a la aplicación real

Una vez validada la viabilidad técnica, el proyecto pasó a una nueva etapa centrada en la aplicación real. A partir de este punto, el objetivo dejó de ser únicamente demostrar que el procesamiento funcionaba y pasó a ser construir un sistema coherente, mantenible y preparado para crecer.

El primer paso consistió en definir los requisitos funcionales, no funcionales y los casos de uso del sistema completo. Esto permitió trabajar desde una visión global y no desde funcionalidades aisladas. A partir de esa visión general se delimitó después el **producto mínimo viable (MVP)**, entendido como un subconjunto coherente del sistema final y no como una simple reducción arbitraria.

Este enfoque permitió centrar la primera iteración en el flujo principal de valor de la aplicación sin comprometer la coherencia del diseño a medio plazo.

5.10. Diseño de datos y persistencia

Una vez definidos los casos de uso, el trabajo avanzó hacia el diseño de la persistencia. Se elaboraron el diagrama Entidad-Relación y el modelo relacional con el objetivo de traducir el comportamiento esperado del sistema a una estructura de datos robusta.

Durante este modelado se siguieron varios criterios: separación clara de conceptos, reducción de redundancia, preparación para el crecimiento, trazabilidad e integridad de negocio. Gracias a ello, el sistema pudo representarse

5.11. MIGRACIÓN DEL ENTORNO DE DESARROLLO A DEBIAN 27

mediante entidades como usuarios, roles, planes, suscripciones, modelos IA, modos, pipelines y ejecuciones.

La principal utilidad de este diseño fue doble: por un lado, servir como documentación; por otro, facilitar una implementación posterior en MariaDB más coherente y menos improvisada.

5.11. Migración del entorno de desarrollo a Debian

Otra decisión relevante fue la migración del entorno de desarrollo a **Debian**. Esta elección respondió a motivos técnicos: mayor cercanía al entorno de despliegue habitual de aplicaciones web, mejor integración con herramientas de backend, tratamiento más consistente de rutas y permisos, y mayor facilidad para la automatización y la administración del sistema.

En el contexto de este TFG, Debian aportó un entorno más natural para trabajar con Python, Flask y MariaDB, además de acercar el desarrollo a un escenario más realista de despliegue.

5.12. Implementación de la base de datos en MariaDB

Con el modelo relacional ya definido y el entorno estabilizado en Debian, se procedió a la implementación inicial de la base de datos en **MariaDB**. Esta fase permitió trasladar al plano físico las decisiones adoptadas en el diseño, mediante claves primarias, restricciones **UNIQUE**, claves foráneas, valores por defecto e integridad transaccional.

La importancia de esta etapa radica en que el proyecto dejó de depender únicamente de la lógica de procesamiento de la demo y pasó a disponer de una base persistente para gestionar usuarios, permisos, configuraciones y ejecuciones de forma coherente.

6. Trabajos relacionados

Este apartado sería parecido a un estado del arte de una tesis o tesina. En un trabajo final grado no parece obligada su presencia, aunque se puede dejar a juicio del tutor el incluir un pequeño resumen comentado de los trabajos y proyectos ya realizados en el campo del proyecto en curso.

7. Conclusiones y Líneas de trabajo futuras

Todo proyecto debe incluir las conclusiones que se derivan de su desarrollo. Éstas pueden ser de diferente índole, dependiendo de la tipología del proyecto, pero normalmente van a estar presentes un conjunto de conclusiones relacionadas con los resultados del proyecto y un conjunto de conclusiones técnicas. Además, resulta muy útil realizar un informe crítico indicando cómo se puede mejorar el proyecto, o cómo se puede continuar trabajando en la línea del proyecto realizado.

Bibliografía

- [1] Atlassian. Jira software documentation. <https://www.atlassian.com/software/jira/guides>, 2024.
- [2] Michael Bayer. Sqlalchemy documentation. <https://www.sqlalchemy.org/>, 2024.
- [3] Google AI Edge. Mediapipe solutions guide. <https://ai.google.dev/edge/mediapipe/solutions/guide>, 2024.
- [4] MariaDB Foundation. Mariadb knowledge base. <https://mariadb.com/kb/en/>, 2024.
- [5] GitHub. Github documentation. <https://docs.github.com/>, 2024.
- [6] Google. Gemini ai. <https://deepmind.google/technologies/gemini/>, 2024.
- [7] Saurabh Kumar. python-dotenv documentation. <https://pypi.org/project/python-dotenv/>, 2024.
- [8] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Abbas Abdulkader, Julian Lhentany, and Matthias Grundmann. Mediapipe: A framework for perceiving and perceiving. *arXiv preprint arXiv:1906.08172*, 2019.
- [9] Microsoft. Microsoft teams documentation. <https://learn.microsoft.com/microsoftteams/>, 2024.
- [10] Microsoft. Visual studio code documentation. <https://code.visualstudio.com/docs>, 2024.

- [11] OpenAI. Chatgpt language model. <https://openai.com/chatgpt>, 2024.
- [12] Debian Project. Debian documentation. <https://www.debian.org/doc/>, 2024.
- [13] Pallets Projects. Flask documentation. <https://flask.palletsprojects.com/>, 2024.
- [14] OpenCV team. Opencv library documentation. <https://docs.opencv.org/>, 2024.
- [15] Landon Turner. Flask-jwt-extended documentation. <https://flask-jwt-extended.readthedocs.io/>, 2024.
- [16] Ultralytics. YOLOv8 docs. <https://docs.ultralytics.com/>, 2024.
- [17] Wikipedia. Latex — wikipedia, la enciclopedia libre. <https://es.wikipedia.org/w/index.php?title=LaTeX&oldid=84209252>, 2015. [Internet; descargado 30-septiembre-2015].